Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach

Jonas Geiping [1] **Sean McLeish** [2] **Neel Jain** [2] **John Kirchenbauer** [2] **Siddharth Singh** [2] **Brian R. Bartoldson** [3]
Bhavya Kailkhura [3] **Abhinav Bhatele** [2] **Tom Goldstein** [2]

Abstract

We study a novel language model architecture that is capable of scaling test-time computation by implicitly reasoning in latent space. Our model works by iterating a recurrent block, thereby unrolling to arbitrary depth at test-time. This stands in contrast to mainstream reasoning models that scale up compute by producing more tokens. Unlike approaches based on chain-of-thought, our approach does not require any specialized training data, can work with small context windows, and can capture types of reasoning that are not easily represented in words. We scale a proof-of-concept model to 3.5 billion parameters and 800 billion tokens. We show that the resulting model can improve its performance on reasoning benchmarks, sometimes dramatically, up to a computation load equivalent to 50 billion parameters.

Model: huggingface.co/tomg-group-umd/huginn-0125
Code and Data: github.com/seal-rg/recurrent-pretraining

1. Scaling by Thinking in Continuous Space

Humans naturally expend more mental effort solving some problems than others. While humans are capable of thinking over long time spans by verbalizing intermediate results and writing them down, a substantial amount of thought happens through complex, recurrent firing patterns in the brain, before the first word of an answer is uttered.

Early attempts at increasing the power of language models focused on scaling model size, a practice that requires extreme amounts of data and computation. More recently, researchers have explored ways to enhance the reasoning capability of models by scaling test time computation. The mainstream approach involves post-training on long chain-of-thought examples to develop the model's ability to *verbalize* intermediate calculations in its context window and thereby externalize thoughts.

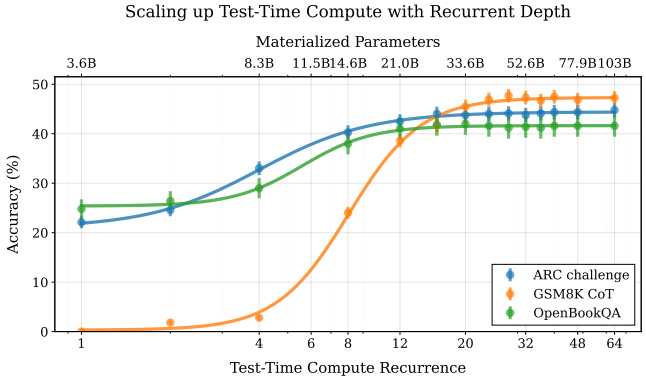

Figure 1: We train a 3.5B parameter language model with depth recurrence. At test time, the model can iterate longer to use more compute and improve its performance. Instead of scaling test-time reasoning by "verbalizing" in long Chains-of-Thought, the model improves entirely by reasoning in latent space. Tasks that require less reasoning like OpenBookQA converge quicker than tasks like GSM8k, which effectively make use of more compute.

However, the constraint that expensive internal reasoning must always be projected down to a single verbalized next token appears wasteful; it is plausible that models could be more competent if they were able to natively "think" in their continuous latent space. One way to unlock this untapped dimension of additional compute involves adding a recurrent unit to a model. This unit runs in a loop, iteratively processing and updating its hidden state and enabling computations to be carried on indefinitely. While this is not currently the dominant paradigm, this idea is foundational to machine learning and has been (re-)discovered in every decade, for example as recurrent neural networks, diffusion models, and as universal or looped transformers.

In this work, we show that depth-recurrent language models can learn effectively, be trained in an efficient manner, and demonstrate significant performance improvements under the scaling of test-time compute. Our proposed trans-

[1]ELLIS Institute Tübingen, Max-Planck Institute for Intelligent Systems, Tübingen AI Center [2]University of Maryland, College Park [3]Lawrence Livermore National Laboratory. Correspondence to: Jonas Geiping, Tom Goldstein <jonas@tue.ellis.eu, tomg@umd.edu>.

former architecture is built upon a latent depth-recurrent block that is run for a randomly sampled number of iterations during training. We show that this paradigm can scale to several billion parameters and over half a trillion tokens of pretraining data. At test-time, the model can improve its performance through recurrent reasoning in latent space, enabling it to compete with other open-source models that benefit from more parameters and training data. Additionally, we show that recurrent depth models naturally support a number of features at inference time that require substantial tuning and research effort in non-recurrent models, such as per-token adaptive compute, (self)-speculative decoding, and KV-cache sharing. We finish out our study by tracking token trajectories in latent space, showing that a number of interesting computation behaviors simply emerge with scale, such as the model rotating shapes in latent space for numerical computations.

2. Why Train Models with Recurrent Depth?

Recurrent layers enable a transformer model to perform arbitrarily many computations before emitting a token. In principle, recurrent mechanisms provide a simple solution for test-time compute scaling. Compared to a more standard approach of long context reasoning (OpenAI, 2024; DeepSeek-AI et al., 2025), latent recurrent thinking has several advantages.

- Latent reasoning does not require construction of bespoke training data. Chain-of-thought reasoning requires the model to be trained on long demonstrations that are constructed in the domain of interest. In contrast, our proposed latent reasoning models can train with a variable compute budget, using standard training data with no specialized demonstrations, and enhance their abilities at test-time if given additional compute.
- Latent reasoning models require less memory for training and inference than chain-of-thought reasoning models. Because the latter require extremely long context windows, specialized training methods such as token-parallelization (Liu et al., 2023a) may be needed.
- Recurrent-depth networks perform more FLOPs per parameter than standard transformers, significantly reducing communication costs between accelerators at scale. This especially enables higher device utilization when training with slower interconnects.
- By constructing an architecture that is *compute-heavy* and small in parameter count, we hope to set a strong prior towards models that solve problems by "thinking", i.e. by learning meta-strategies, logic and abstraction, instead of memorizing. The strength of recurrent priors for learning complex algorithms has already been demonstrated in the "deep thinking" literature (Schwarzschild et al., 2021b; Bansal et al., 2022; Schwarzschild et al., 2023).

On a more philosophical note, we hope that latent reasoning captures facets of human reasoning that defy verbalization, such as spatial thinking, physical intuition or (motor) planning. Over many iterations of the recurrent process, reasoning in a high-dimensional vector space would enable the deep exploration of multiple directions simultaneously, instead of linear thinking, leading to a system capable of exhibiting novel and complex reasoning behavior.

Scaling compute in this manner is not at odds with scaling through extended (verbalized) inference scaling (Shao et al., 2024), or scaling parameter counts in pretraining (Kaplan et al., 2020), we argue it may build a third axis on which to scale model performance.

Table of Contents

3. A scalable recurrent architecture

In this section we will describe our proposed architecture for a transformer with latent recurrent depth, discussing design choices and small-scale ablations. A diagram of the architecture can be found in Figure 2. We always refer to the sequence dimension as n, the hidden dimension of the model as h, and its vocabulary as the set V.

3.1. Macroscopic Design

The model is primarily structured around decoder-only transformer blocks (Vaswani et al., 2017; Radford et al., 2019). However these blocks are structured into three functional groups, the *prelude* P, which embeds the input data into a latent space using multiple transformer layers, then the core *recurrent block* R, which is the central unit of recurrent computation modifying states $\mathbf{s} \in \mathbb{R}^{n \times h}$, and finally the *coda* C, which un-embeds from latent space using several layers and also contains the prediction head of the model. The core block is set between the prelude and coda blocks, and by looping the core we can put an indefinite amount of verses in our song.

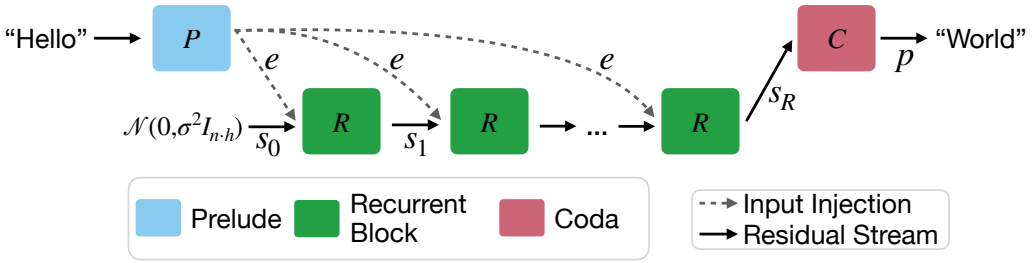

Figure 2: A visualization of the Architecture, as described in Section 3. Each block consists of a number of sub-layers. The blue prelude block embeds the inputs into latent space, where the green shared *recurrent block* is a block of layers that is repeated to compute the final latent state, which is decoded by the layers of the red coda block.

Given a number of recurrent iterations r, and a sequence of input tokens $\mathbf{x} \in V^n$ these groups are used in the following way to produce output probabilities $\mathbf{p} \in \mathbb{R}^{n \times |V|}$

$$\begin{aligned} \mathbf{e} &= P(\mathbf{x}) \\ \mathbf{s}_0 &\sim \mathcal{N}(\mathbf{0}, \sigma^2 I_{n\cdot h}) \\ \mathbf{s}_i &= R(\mathbf{e}, \mathbf{s}_{i-1}) \qquad \text{for} \quad i \in \{1, \dots, r\} \\ \mathbf{p} &= C(\mathbf{s}_r), \end{aligned}$$

where σ is some standard deviation for initializing the random state. This process is shown in Figure 2. Given an init random state $\mathbf{s}_0$, the model repeatedly applies the core block R, which accepts the latent state $\mathbf{s}_{i-1}$ and the embedded input $\mathbf{e}$ and outputs a new latent state $\mathbf{s}_i$. After finishing all iterations, the coda block processes the last state and produces the probabilities of the next token.

This architecture is based on deep thinking literature, where it is shown that injecting the latent inputs $\mathbf{e}$ in every step (Bansal et al., 2022) and initializing the latent vector with a random state stabilizes the recurrence and promotes convergence to a steady state independent of initialization, i.e. *path independence* (Anil et al., 2022).

Motivation for this Design. This recurrent design is the minimal setup required to learn stable iterative operators. A good example is gradient descent of a function $E(\mathbf{x}, \mathbf{y})$, where $\mathbf{x}$ may be the variable of interest and $\mathbf{y}$ the data. Gradient descent on this function starts from an initial random state, here $\mathbf{x}_0$, and repeatedly applies a simple operation (the gradient of the function it optimizes), that depends on the previous state $\mathbf{x}_k$ and data $\mathbf{y}$. Note that we need to use $\mathbf{y}$ in every step to actually optimize our function. Similarly we repeatedly inject the data $\mathbf{e}$ in our set-up in every step of the recurrence. If $\mathbf{e}$ was provided only at the start, e.g. via $\mathbf{s}_0 = \mathbf{e}$, then the iterative process would not be stable[1], as its solution would depend only on its boundary conditions.

The structure of using several layers to embed input tokens into a hidden latent space is based on empirical results analyzing standard fixed-depth transformers (Skean et al., 2024; Sun et al., 2024; Kaplan et al., 2024). This body of research shows that the initial and the end layers of LLMs are noticeably different, whereas middle layers are interchangeable and permutable. For example, Kaplan et al. (2024) show that within a few layers standard models already embed sub-word tokens into single concepts in latent space, on which the model then operates.

Remark 3.1 (Is this a Diffusion Model?). This iterative architecture will look familiar to the other modern iterative modeling paradigm, diffusion models (Song and Ermon, 2019), especially latent diffusion models (Rombach et al., 2022). We ran several ablations with iterative schemes even more similar to diffusion models, such as $\mathbf{s}_i = R(\mathbf{e}, \mathbf{s}_{i-1}) + \mathbf{n}$ where $\mathbf{n} \sim \mathcal{N}(\mathbf{0}, \sigma_i I_{n\cdot h})$, but find the injection of noise not to help in our preliminary experiments, which is possibly connected to our training objective. We also evaluated and $\mathbf{s}_i = R_i(\mathbf{e}, \mathbf{s}_{i-1})$, i.e. a core block that takes the current step as input (Peebles and Xie, 2023), but find that this interacts badly with path independence, leading to models that cannot extrapolate.

3.2. Microscopic Design

Within each group, we broadly follow standard transformer layer design. Each block contains multiple layers, and each layer contains a standard, causal self-attention block using RoPE (Su et al., 2021) with a base of 50000, and a gated SiLU MLP (Shazeer, 2020). We use RMSNorm (Zhang and Sennrich, 2019) as our normalization function. The model has learnable biases on queries and keys, and nowhere else. To stabilize the recurrence, we order all layers in the following "sandwich" format, using norm layers n_i, which is related, but not identical to similar strategies in (Ding et al., 2021; Team Gemma et al., 2024):

$$\begin{aligned} \hat{\mathbf{x}}_l &= n_2\left(\mathbf{x}_{l-1} + \text{Attn}(n_1(\mathbf{x}_{l-1}))\right) \\ \mathbf{x}_l &= n_4\left(\hat{\mathbf{x}}_l + \text{MLP}(n_3(\hat{\mathbf{x}}_l))\right) \end{aligned}$$

While at small scales, most normalization strategies, e.g. pre-norm, post-norm and others, work almost equally well,

[1] Stable in the sense that R cannot be a monotone operator if it does not depend on $\mathbf{e}$, and so cannot represent gradient descent on strictly convex, data-dependent functions, (Bauschke et al., 2011)